```
lm.create_monitor(
    table_name=f"{catalog}.{database_name}.packaged_transaction_model_predictions",
    profile_type=lm.InferenceLog(
        timestamp_col="TransactionTimestamp",
        granularities=["5 minutes","30 minutes","1 hour"],
        model_id_col="model_version",
        prediction_col="prediction",
        label_col="actual_label",
        problem_type="classification"
    ),
    slicing_exprs=["Product","CustomerID"],
    schedule=lm.MonitorCronSchedule(
        quartz_cron_expression="0 0 * * * ?", # schedules a refresh every hour
        timezone_id="MST",
    ),
    output_schema_name=f"{catalog}.{database_name}"
)
```

Figure 8.28 – The inference table monitor

Congratulations! The streaming project is complete. We encourage you to add improvements and commit them back to the repository. Here are a few possible examples: add more validation metrics to the validation notebook, incorporate the inference performance results into the decision to retrain, and make adjustments to the configuration of data generation to simulate drift.

Project: Retrieval-Augmented Generation Chatbot

To follow along in your own workspace, please open the following notebooks and resources:

- CH8-app.py
- CH8-01-Deploy Your Endpoint with SDK
- The **Hugging Face Spaces** page

First, we need to ensure our chatbot is deployed using Model Serving, as shown in *Figure 8.30*. Here, we are using the fastest way by going through the UI of the Model Serving page. To follow along and serve, we are selecting the registered model that we registered in *Chapter 7*. Select the latest version – in our case, it's version 4. For this demo project, we expect minimal concurrency so the endpoint will have only four possible concurrent runs and will scale to 0 when no traffic is available. We are enabling inference tables under the same catalog to track and potentially further monitor our payload. We are not going to demonstrate in this chapter how to set a monitor or data quality pipeline for the RAG project, as it was demonstrated for the streaming project. We encourage you to apply it on your own!